

EXISTING SYSTEM

The existing research process mainly depends on traditional search engines, academic databases, online articles, and general-purpose AI-based question-answering systems. When a user has a research question, the user generally enters keywords into a search engine and receives a list of webpages, articles, or research papers. The user must then manually open the relevant sources, read the available information, identify useful content, compare information from different sources, and prepare the final research output.

Academic search platforms can provide access to research papers and scholarly information, but the process of selecting relevant papers, reading them, extracting useful information, and comparing their findings still requires significant manual effort. Similarly, general AI assistants can provide quick answers to user questions, but they may not always perform systematic multi-step research across different sources. The user may still need to verify the generated information and search for supporting references separately.

Limitations of the Existing System

- Requires significant manual effort for searching and reading information.
- Information is collected from multiple sources separately.
- Complex research questions are not automatically divided into sub-questions.
- Users must manually verify and compare information from different sources.
- No dedicated agent automatically evaluates research completeness.
- Additional research must generally be performed manually when information is insufficient.
- Final reports and citations often need to be prepared manually.
- Traditional systems do not provide an integrated multi-agent research workflow.

PROPOSED SYSTEM

The proposed Agentic Deep Researcher is an AI-based multi-agent research system designed to automate and improve the process of deep research.

When the user enters a research question, the Planner Agent first analyzes the question and divides it into meaningful sub-questions. Based on the research plan, the Searcher Agent generates appropriate search queries and retrieves relevant information from different sources. External research tools such as Tavily, Exa, and Semantic Scholar can be used to obtain web-based and academic information. The Fetcher Agent then retrieves and processes the content from the selected sources. The collected information can be organized using Retrieval-Augmented Generation (RAG) and a knowledge store such as ChromaDB, allowing relevant information to be retrieved when required. The Critic Agent evaluates the collected research to determine whether the information is relevant, sufficient, and covers the important aspects of the research question, if it identifies missing information or research gaps, the system can send the task back to the Searcher Agent for additional research. This creates an iterative research loop that allows the system to improve the quality and completeness of the collected information. Finally the Report Generation module prepares a structured final report containing the major findings, source references, inline citations, and confidence information.

The workflow can be coordinated using frameworks such as LangGraph ,CrewAI, while Model Context Protocol (MCP) can provide a standardized connection between the AI agents and external tools. An LLM, such as a locally hosted model through Ollama, can support reasoning, planning, evaluation, and report generation.

Key Features

- Automatic decomposition of complex research questions.
- Multi-agent research using specialized Planner, Searcher, Fetcher, and Critic Agents.
- Integration with web and academic search tools.
- Iterative research when information gaps are detected.
- Retrieval and organization of relevant information using RAG.
- Structured report generation with citations and confidence information.
- Reduced manual effort in collecting and organizing research information.

HARDWARE REQUIREMENTS

The Agentic Deep Researcher requires a computer system capable of running Python-based applications, AI models, agent workflows, and information retrieval components. Since the system may use a locally hosted Large Language Model (LLM), adequate processing power and memory are required for smooth execution.

Hardware Component	Minimum Requirement	Recommended Requirement
Processor	Intel Core i5 / AMD Ryzen 5	Intel Core i5/i7 or AMD Ryzen 5/7
RAM	8 GB	16 GB or higher
Storage	10 GB free space	20 GB or more SSD space
Display	1366 × 768 resolution	Full HD (1920 × 1080)
Internet	Stable internet connection	High-speed broadband/Wi-Fi
Operating System	64-bit Windows/Linux/macOS	Windows 10/11 64-bit or equivalent
Input Devices	Keyboard and mouse	Standard keyboard and mouse

Overall, the system can be developed and tested on a standard modern laptop or desktop with 8 GB RAM, an Intel Core i5/Ryzen 5 processor, sufficient SSD storage, and a stable Internet connection. Higher specifications are recommended for efficient local LLM execution and larger research workloads.

SOFTWARE REQUIREMENTS

The Agentic Deep Researcher requires a software environment that supports Python-based development, Large Language Models, multi-agent orchestration, information retrieval, knowledge storage, and document generation. The major software components required for developing and executing the proposed system are given below.

Software Technology	Requirement / Purpose
Operating System	Windows 10/11 64-bit, Linux, or macOS
Python	Python 3.10 or above for application development
IDE / Code Editor	Visual Studio Code or similar development environment
LLM	Local or API-based Large Language Model for reasoning and generation
Ollama	Used to run supported LLMs locally
LangGraph CrewAI	Agent workflow and multi-agent orchestration
MCP	Connects AI agents with external tools and services
Tavily	Web search and information retrieval
Exa	AI-oriented web search and retrieval
Semantic Scholar	Academic paper and research information retrieval
RAG	Retrieval-Augmented Generation for using retrieved information
ChromaDB	Vector database for storing and retrieving embeddings
Git / GitHub	Version control and project management
Web Browser	Accessing research sources and testing the application